



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category – I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Assignment No. 05

Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	Adarsh Jha
PRN No.	23070122261
Branch	CSE, Batch (2023-27), L4
Academic Year & Semester	25 – 26, Sem VII
Date of Performance	16 th July, 2026
Title of Assignment:	Continuous Integration with Jenkins
Theory:	Theory Jenkins is an open-source automation server used to implement Continuous Integration (CI) and Continuous Delivery (CD). It automates software development tasks such as building, testing, and deploying applications. Jenkins integrates with version control systems like Git to automatically retrieve source code and execute predefined build steps. In this experiment, Jenkins was installed using Docker, configured with the Git plugin, and connected to a GitHub repository. A Freestyle project was created to pull source code from the repository and execute shell commands. The build was then triggered manually, and the console output was examined to verify successful execution. Commands Used • docker --version – Verified that Docker was installed on the system. • docker ps – Checked whether the Jenkins Docker container was running. • docker run -d -p 8080:8080 -p 50000:50000 -v jenkins_home:/var/jenkins_home --name myjenkins jenkins/jenkins:lts-jdk17 – Created and started the Jenkins Docker container. • docker logs myjenkins – Displayed the Jenkins logs to obtain the initial administrator password during the initial setup.

	• docker exec myjenkins cat /var/jenkins_home/secrets/initialAdminPassword – Retrieved the initial administrator password generated by Jenkins. Execute Shell Commands • echo "BUILDING: \${JOB_NAME}" – Displayed the Jenkins job name. • echo "BUILD NUMBER: \${BUILD_NUMBER}" – Displayed the current build number. • echo "WORKSPACE: \${WORKSPACE}" – Displayed the Jenkins workspace path. • ls -la – Listed all files present in the Jenkins workspace. • echo "Hello Jenkins!" – Displayed a sample message to verify successful execution of the build step.
Implementation:	Step 1: Verify Docker and Jenkins Container Verified Docker and Jenkins Container — Verified Docker installation and confirmed that the Jenkins Docker container (myjenkins) was running successfully. Step 2: Access Jenkins Dashboard Accessed Jenkins Dashboard — Successfully accessed the Jenkins Dashboard and verified that the existing Freestyle project (Git-Practice-Build) was available. Step 3: Verify Git Plugin Installation Verified Git Plugin — Verified that the Git plugin was installed and enabled, allowing Jenkins to integrate with Git repositories for source code management. Step 4: Open the Freestyle Project Opened Freestyle Project — Opened the existing Freestyle project (Git-Practice-Build) to verify its configuration and prepare it for executing the build process. Step 5: Verify Source Code Management (Git) Configured Source Code Management — Configured the Freestyle project to retrieve source code from the GitHub repository by selecting Git as the Source Code Management option and specifying the repository URL. Step 6: Verify Build Step Configured Build Step — Verified the Execute shell build step containing shell commands to display build information, list the repository files, and print a sample message during the build process. Step 7: Save and Run the Build Executed Build — Successfully executed the Freestyle project using Build Now and verified that the build completed successfully with a SUCCESS status.

Step 8: View Console Output

Verified Console Output — Examined the console output and verified that Jenkins successfully cloned the Git repository, executed the configured shell commands, and completed the build with a SUCCESS status.

```
Command Prompt

Microsoft Windows [Version 10.0.26200.9457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Adarsh Jha>docker --version
Docker version 29.1.3, build f52814d

C:\Users\Adarsh Jha>
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
cfd8dc610bc2	jenkins/jenkins:lts	"/usr/bin/tini -- /u..."	2 days ago	Up 8 minutes	0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp, 0.0.0.0:50000->50000/tcp, [::]:50000->50000/tcp

Jenkins

All

New Item

Enter an item name

Flask-HelloWorld-Freestyle

Select an item type

Pipeline

Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.

Freestyle project

Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.

Multi-configuration project

Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.

Folder

Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.

Multibranch Pipeline

Creates a set of Pipeline projects according to detected branches in one SCM repository.

OK

Jenkins

Flask-HelloWorld-Freestyle

#2

Console Output

Delete build '92'

Git Build Data

Edit build information

Started by user: Adarsh Jha

Running as SYSTEM

Building in workspace /var/jenkins_home/workspace/Flask-HelloWorld-Freestyle

The recommended git tool is: NONE

No credentials specified

> git rev-parse --resolve-git-dir /var/jenkins_home/workspace/Flask-HelloWorld-Freestyle/.git # timeout=10

Fetching changes from the remote Git repository

> git config remote.origin.url https://github.com/AdarshCodes1221/DevOps-Lab-Assignments.git # timeout=10

Fetching upstream changes from https://github.com/AdarshCodes1221/DevOps-Lab-Assignments.git

> git --version # timeout=10

> git --version # 'git version 2.47.3'

> git fetch --tags --force --progress -- https://github.com/AdarshCodes1221/DevOps-Lab-Assignments.git +refs/heads/*:refs/remotes/origin/* # timeout=10

> git rev-parse refs/remotes/origin/main{commit} # timeout=10

Checking out revision 65f02dd9b6ef6520b43d80a1b2254f553b843c5 (refs/remotes/origin/main)

> git config core.sparsecheckout # timeout=10

> git checkout -f 65f02dd9b6ef6520b43d80a1b2254f553b843c5 # timeout=10

Commit message: "Resolved merge conflict"

First time build. Skipping changelog.

[Flask-HelloWorld-Freestyle] \$ /bin/sh -xe /tmp/jenkins13643275525767261008.sh

+ echo Listing workspace files

Listing workspace files

+ ls -la

total 20

drwxr-xr-x 5 jenkins jenkins 4096 Jul 27 08:51 .

drwxr-xr-x 8 jenkins jenkins 4096 Jul 27 08:50 ..

drwxr-xr-x 8 jenkins jenkins 4096 Jul 27 08:51 .git

drwxr-xr-x 2 jenkins jenkins 4096 Jul 27 08:51 Assignment-TM1.1-Git

drwxr-xr-x 2 jenkins jenkins 4096 Jul 27 08:51 Screenshots

Finished: SUCCESS

Subject Faculty Name: Prof. Arijit Dutta